

Project Specification: Industrial IoT Telemetry Pipeline

Travis Asher

2026-08-10

- Industrial IoT Telemetry Pipeline: A Dual-Engine Performance Study
 - Project Scenario and Data Profile
 - Raw Data Ingestion Profile
 - Technical Requirements and Phases
 - Phase 1: Ingestion, Header Normalization, and Token Extraction
 - Phase 2: Priority Conditionals and Risk Profile Classification
 - Phase 3: Spatial Filtering and Anomaly Data Cleansing
 - Phase 4: Partitioned Analytics and Windowed Voltage Ranking
 - Phase 5: Query Plan Inspection and Lazy Engine Performance Audit

Industrial IoT Telemetry Pipeline: A Dual-Engine Performance Study

An end-to-end data engineering case study evaluating text parsing, data sanitation, and partitioned analytics executed in parallel across both Pandas (Eager) and Polars (Lazy) DataFrame objects.

Project Scenario and Data Profile

A regional smart-grid sensor network emits unstructured log strings containing mixed data types, missing coordinates, and malformed identifiers. The objective is to clean the tracking keys, extract metrics using regular expressions, classify operational risk thresholds, and compute isolated rank metrics per zone.

Raw Data Ingestion Profile

The following raw data collection initializes the pipeline's source streams:

```
import numpy as np
import pandas as pd
import polars as pl

sensor_data = {
    "SENSOR_ID": [
        "sn-101 ", "sn-102", "sn-103 ", "SN-101", "sn-104",
        "sn-102 ", "sn-105 ", "sn-101", "SN-103", "sn-104 ",
        "sn-105", "sn-102", "sn-101 ", "sn-103 ", "sn-104"
    ],
    "raw_logs": [
        "ERROR|LOC:40.7128,-74.0060|VOLT:240.5",
        "WARN|LOC:39.9526,-75.1652|VOLT:missing",
        "OK|LOC:34.0522,-118.2437|VOLT:120.2",
        "ERROR|LOC:40.7128,-74.0060|VOLT:285.0",
        "OK|LOC:41.8781,-87.6298|VOLT:118.9",
        "ERROR|LOC:39.9526,-75.1652|VOLT:310.1",
        "WARN|LOC:null|VOLT:190.4",
        "OK|LOC:40.7128,-74.0060|VOLT:238.1",
        "WARN|LOC:34.0522,-118.2437|VOLT:244.9",
        "ERROR|LOC:41.8781,-87.6298|VOLT:262.3",
        "OK|LOC:45.5017,-73.5673|VOLT:121.5",
        "OK|LOC:39.9526,-75.1652|VOLT:220.4",
        "WARN|LOC:40.7128,-74.0060|VOLT:255.2",
        "ERROR|LOC:34.0522,-118.2437|VOLT:missing",
        "WARN|LOC:41.8781,-87.6298|VOLT:249.8"
    ],
    "timestamp": [
        "2026-08-01 10:00", "2026-08-01 10:05", "2026-08-01 10:10", "2026-08-01 10:15", "2026-08-01 10:20",
        "2026-08-01 10:25", "2026-08-01 10:30", "2026-08-01 10:35", "2026-08-01 10:40", "2026-08-01 10:45",
        "2026-08-01 10:50", "2026-08-01 10:55", "2026-08-01 11:00", "2026-08-01 11:05", "2026-08-01 11:10"
    ]
}
```

Technical Requirements and Phases

All transformations must be executed using a single, unbroken method chain for Pandas, and a single, unbroken expression query plan for Polars.

Phase 1: Ingestion, Header Normalization, and Token Extraction

- Standardize column headers to lowercase and strip all structural whitespace.
- Clean up `sensor_id` strings to establish a uniform uppercase format (`"SN-101"`) with no trailing padding.
- Parse the `raw_logs` string column using Regular Expressions to extract three clean features:
 - `status` : The alphabetic string preceding the first pipe (`|`) delimiter.
 - `latitude` : The numeric coordinate text isolated immediately following the `LOC:` flag.
 - `voltage` : The floating-point decimal trailing the `VOLT:` flag, coercing non-numeric text like `"missing"` into true null values.
 - `timestamp` : The raw temporal text string coerced into a native datetime object.

Phase 2: Priority Conditionals and Risk Profile Classification

- Derive a new categorical attribute named `risk_profile` based on the operational thresholds:
 - If `status` is `"ERROR"` and `voltage` exceeds `250.0` -> label it `"CRITICAL"` .
 - If `status` is `"ERROR"` or `"WARN"` -> label it `"HIGH"` .
 - Otherwise -> label it `"STABLE"` .
 - Engine Constraints:** Pandas must execute this vectorization step within `.assign()` via `np.select()` . Polars must utilize native `pl.when().then().otherwise()` syntax.

Phase 3: Spatial Filtering and Anomaly Data Cleansing

- Filter out and drop all rows containing missing values or null entries inside the extracted `latitude` and `voltage` columns.

Phase 4: Partitioned Analytics and Windowed Voltage Ranking

- Compute a relative ranking metric across the dataset while fully preserving the row density of individual log records (no grouping or structural row reduction):
 - Create a column named `voltage_rank_in_sensor` that ranks the row's voltage relative to that specific `sensor_id` , ordered from highest voltage to lowest voltage.
 - Engine Constraints:** Pandas must leverage `.groupby().rank()` . Polars must use an expression block evaluated over the partition via `.over()` .
 - Output a Polars DataFrame object that displays the number of each unique `sensor_id` values. **Engine Constraints:** NOT required to be part of the single expression query chain

Phase 5: Query Plan Inspection and Lazy Engine Performance Audit

- Evaluate the compiled query graph of the finalized Polars pipeline.
 - Output the optimized logical execution tree to the terminal console using the `.explain()` inspector to audit predicate pushdowns executed by the Polars optimization tree.
 - Output the full, final Polars DataFrame.